

05_tendencias_top20

May 11, 2026

1 Fase 3: Análisis de Tendencias Top 20

Este cuaderno tiene un enfoque descriptivo y busca responder: “¿Qué tecnologías dominan en volumen absoluto?” basándose en los datos de Stack Overflow.

1.1 1. Carga, Filtrado y Conteo Total por Tag

A continuación importamos las librerías necesarias, incluidas las funciones estandarizadas de nuestro módulo `src.metrics`. Posteriormente, cargamos los datos procesados, filtramos las preguntas correspondientes al período **2015–2024** y calculamos el volumen absoluto para obtener las 20 tecnologías principales.

```
[1]: import polars as pl
import plotly.express as px
import sys
import os

# Añadir directorio raíz al sys.path para importar src
sys.path.append(os.path.abspath('../'))
from src.metrics import calculate_relative_percentage, categorize_technology

# Rutas de los datos procesados
DIM QUESTIONS_PATH = '../data/datos_procesados/dim_questions.parquet'
FACT_TAGS_PATH = '../data/datos_procesados/fact_question_tags.parquet'
TOP20_OUTPUT_PATH = '../data/datos_procesados/eda/top20.parquet'
```

1.1.1 Carga y Auditoría del Filtrado

Cargamos el dataset de preguntas, registramos el volumen inicial de datos y filtramos únicamente los años entre 2015 y 2024. Posteriormente reportamos el número de filas después del filtro, tal como se requiere en la auditoría.

```
[2]: # 1. Cargar las preguntas usando evaluación lazy para eficiencia
questions_lazy = pl.scan_parquet(DIM_QUESTIONS_PATH)

# Calcular total antes del filtro
total_antes = questions_lazy.select(pl.len()).collect().item()

# Aplicar el filtro de fecha (2015-2024) extrayendo el año de CreationDate
```

```

questions_filtradas = questions_lazy.filter(
    (pl.col('CreationDate').dt.year() >= 2015) &
    (pl.col('CreationDate').dt.year() <= 2024)
)

# Calcular total después del filtro
total_despues = questions_filtradas.select(pl.len()).collect().item()

print(f"--- Auditoría de Filtrado ---")
print(f"Filas ANTES del filtro de fecha: {total_antes:,}")
print(f"Filas DESPUÉS del filtro de fecha: {total_despues:,}")
print(f"Porcentaje de retención: {(total_despues / total_antes) * 100:.2f}%")

--- Auditoría de Filtrado ---
Filas ANTES del filtro de fecha: 16,055,694
Filas DESPUÉS del filtro de fecha: 16,055,694
Porcentaje de retención: 100.00%

```

```

[3]: from narwhals import LazyFrame
      dataf = pl.scan_parquet(FACT_TAGS_PATH)

```

1.1.2 Cálculo del Top 20

Realizamos un JOIN entre las preguntas filtradas y sus etiquetas asociadas. Luego agrupamos por Tag, contamos la cantidad de ocurrencias (Id) ordenando de forma descendente, y extraemos el Top 20.

```

[4]: # Cargar tabla de hechos de tags de forma lazy
      tags_lazy = pl.scan_parquet(FACT_TAGS_PATH).rename({"Tags": "Tag"})

      # Join entre preguntas filtradas y tags, y luego agrupación para calcular el
      ↪ volumen total
      top20_df = (
          questions_filtradas
            .join(tags_lazy, on='Id', how='inner')
            .group_by('Tag')
            .agg(pl.len().alias('Count'))
            .sort('Count', descending=True)
            .head(20)
      ).collect()

      print("Top 20 Tecnologías por volumen absoluto:")
      display(top20_df)

```

Top 20 Tecnologías por volumen absoluto:

shape: (20, 2)

Tag	Count
-----	-------

```

---
str          u32

python       1830210
javascript   1808537
java         1186673
c#           909857
android      858772
...
mysql        385320
python-3.x   327379
swift        317837
angular      303776
arrays       300565

```

1.2 2. Segmentación por Categoría

Utilizamos nuestra función estandarizada `categorize_technology` del módulo `src.metrics` para asignar cada uno de los tags obtenidos a una categoría específica: “Lenguaje”, “Framework”, “Base de Datos” u “Otros”.

```

[5]: # Aplicar la función de categorización
top20_df = top20_df.with_columns(
    categorize_technology('Tag', 'Categoria')
)

display(top20_df)

```

shape: (20, 3)

Tag	Count	Categoria
---	---	---
str	u32	str
python	1830210	Lenguaje
javascript	1808537	Lenguaje
java	1186673	Lenguaje
c#	909857	Lenguaje
android	858772	Otros
...	...	...
mysql	385320	Base de Datos
python-3.x	327379	Otros
swift	317837	Lenguaje
angular	303776	Framework
arrays	300565	Otros

1.3 3. Participación Relativa

Procedemos a calcular el porcentaje que representa cada tag del Top 20 sobre el **total absoluto de preguntas del período 2015-2024**. Usaremos el `total_despues` calculado anteriormente como nuestro universo de preguntas (denominador). Importamos nuestra función `calculate_relative_percentage` de `src.metrics`.

```
[6]: # Calcular el porcentaje relativo
top20_df = calculate_relative_percentage(
    df=top20_df,
    count_col='Count',
    total_count=total_despues,
    percentage_col_name='Porcentaje'
)

display(top20_df)
```

shape: (20, 4)

Tag	Count	Categoria	Porcentaje
---	---	---	---
str	u32	str	f64
python	1830210	Lenguaje	11.399134
javascript	1808537	Lenguaje	11.264147
java	1186673	Lenguaje	7.390979
c#	909857	Lenguaje	5.666881
android	858772	Otros	5.348707
...	...	...	...
mysql	385320	Base de Datos	2.399896
python-3.x	327379	Otros	2.039021
swift	317837	Lenguaje	1.979591
angular	303776	Framework	1.892014
arrays	300565	Otros	1.872015

1.4 4. Visualizaciones Requeridas

A continuación, construiremos tres visualizaciones interactivas usando Plotly para presentar el análisis descriptivo: 1. Un gráfico de barras horizontales mostrando las 20 tecnologías de mayor volumen. 2. Tres gráficos de barras detallando el volumen interno por categoría. 3. Un Treemap representando la participación relativa general.

```
[7]: # 1. Gráfico de barras horizontales: Top 20 por volumen absoluto
fig1 = px.bar(
    top20_df.to_pandas().sort_values('Count', ascending=True), # Sort asc para
    que Plotly muestre los mayores arriba
    x='Count',
    y='Tag',
```

```

orientation='h',
color='Categoria',
title='Top 20 Tecnologías por Volumen Absoluto (2015-2024)',
labels={'Count': 'Volumen de Preguntas', 'Tag': 'Tecnología', 'Categoria': 'Categoría'},
color_discrete_sequence=px.colors.qualitative.Pastel
)

fig1.update_layout(
    template='plotly_white',
    height=600,
    title_font_size=20,
    margin=dict(l=100, r=20, t=60, b=20)
)
fig1.show()

```

1.4.1 Volumen Interno por Categoría

Análisis detallado de la distribución de tecnologías agrupadas por las 3 categorías principales detectadas en el Top 20.

```

[8]: # 2. Gráficos de barras por categoría
df_pd = top20_df.to_pandas()

for categoria in ['Lenguaje', 'Framework', 'Base de Datos']:
    df_cat = df_pd[df_pd['Categoria'] == categoria].sort_values('Count',
    ↪ascending=False)

    if df_cat.empty:
        continue

    fig = px.bar(
        df_cat,
        x='Tag',
        y='Count',
        title=f'Volumen Absoluto - Categoría: {categoria}',
        labels={'Count': 'Volumen de Preguntas', 'Tag': 'Tecnología'},
        color='Tag',
        text_auto='.2s'
    )

    fig.update_layout(
        template='plotly_white',
        showlegend=False,
        height=400
    )

```

```
fig.update_traces(textfont_size=12, textangle=0, textposition="outside",
↪cliponaxis=False)
fig.show()
```

1.4.2 Gráfico de Pareto de Participación Relativa

Este gráfico permite visualizar el porcentaje relativo de cada tecnología y también el total acumulado, nos cuenta la historia de la dominancia de las tecnologías en el ecosistema.

```
[9]: import plotly.graph_objects as go

# 1. Preparar los datos ordenados de mayor a menor porcentaje
df_pd = top20_df.to_pandas().sort_values('Porcentaje', ascending=False)

# 2. Calcular el porcentaje acumulado
df_pd['Porcentaje_Acumulado'] = df_pd['Porcentaje'].cumsum()

# 3. Crear la figura base
fig3 = go.Figure()

# 4. Añadir las barras de porcentaje individual
fig3.add_trace(go.Bar(
    x=df_pd['Tag'],
    y=df_pd['Porcentaje'],
    name='Participación Individual (%)',
    marker_color='#316395', # Un azul sobrio y profesional
    text=df_pd['Porcentaje'].apply(lambda x: f'{x:.1f}%'),
    textposition='auto'
))

# 5. Añadir la línea de porcentaje acumulado (Eje Secundario)
fig3.add_trace(go.Scatter(
    x=df_pd['Tag'],
    y=df_pd['Porcentaje_Acumulado'],
    name='Porcentaje Acumulado',
    mode='lines+markers',
    marker=dict(color='#DC3912', size=8), # Rojo para contraste
    line=dict(width=3),
    yaxis='y2'
))

# 6. Configurar el diseño y el doble eje Y
fig3.update_layout(
    title='Concentración del Ecosistema: Participación Individual y Acumulada',
    template='plotly_white',
    height=600,
    margin=dict(t=60, l=40, r=40, b=40),
```

```

legend=dict(x=0.65, y=0.1, bgcolor='rgba(255,255,255,0.8)'),
yaxis=dict(
    title='Porcentaje de Preguntas (%)',
    showgrid=False
),
yaxis2=dict(
    title='Porcentaje Acumulado (%)',
    overlaying='y',
    side='right',
    range=[0, 100], # El acumulado siempre llega a 100 o cerca
    showgrid=True,
    gridcolor='#E5ECF6'
)
)

fig3.show()

```

1.5 5. Exportación del Entregable

Procedemos a guardar el dataset limpio de Top 20, ya categorizado y con porcentajes de mercado calculados. Este archivo en formato parquet alimentará posteriormente los tableros de control (Dashboard) y la API del producto.

```

[10]: # Exportar el entregable en formato parquet
# Creamos el directorio si no existe (aunque dado que leímos de ahí asume su
      ↪ existencia)
os.makedirs(os.path.dirname(TOP20_OUTPUT_PATH), exist_ok=True)

top20_df.write_parquet(TOP20_OUTPUT_PATH)
print(f"Archivo exportado con éxito en: {TOP20_OUTPUT_PATH}")

```

Archivo exportado con éxito en: ../data/datos_procesados/eda/top20.parquet

1.6 6. Contraste con Hipótesis

A continuación, contrastamos estos hallazgos puramente descriptivos con nuestras hipótesis originales definidas en la **Fase 1**.

1.6.1 4. Contraste inicial con hipótesis

[PLACEHOLDER PARA EL USUARIO]:

Redacta aquí cómo el ranking real (el volumen y las tecnologías dominantes observadas en los gráficos) confirma o refuta las hipótesis que se plantearon en la Fase 1. Analiza el papel de las tecnologías emergentes frente a los líderes del mercado.